

Introduction

The consumer's need for accurate representation of audio program material is much less strict than that of professional listeners. Many algorithms have been developed to carefully model a signal and throw out less important information based on users' preferences and tastes. They are called *lossy* algorithms. There are similar algorithms for images too. We need a different type of compression algorithm to cover the case where any loss of information would be unacceptable, whether that be for technical or aesthetic reasons. These are called *lossless* algorithms.

This paper focuses on audio signals used in the entertainment industry such as music compact disks (CDs) and movie sound tracks. In general, a signal can be anything that represents something in the real world; a sonogram, radar, radio, video, or even still pictures. Lossless compression happens when a pattern can be described with symbols that are only as big as needed at a given instant to uniquely and exactly represent each value. Signals are a special case, as the number of bits needed to represent a value are often as high as 32 bits (sometimes more), rather than the typical eight used for text and executables, and also have the property that each instance (or symbol) is closely related to the next. And, where text and executables are precise, a repeat of the same character or word is exactly the same character or word. A signal can be say, a half-decibel quieter than the original (values approximately 94% of the magnitude of the original) and not be discernable from the original signal. Even trained listeners without a side-by-side comparison won't be able to detect this change. Signal compression algorithms go this extra step to find the similarities between adjacent samples and only store the differences between what the real value is and what some prediction scheme says the value should be.

These similarities can be calculated with some very complicated digital signal processing (DSP) algorithms.

The consumer marketplace has shown a desire for storing large amounts of audio. CD quality digital audio files are about 10.5 megabytes in size for every minute of audio. This means there's a need for 500 to 700 megabytes of storage for each CD in a collection. Doing the math, a portable player's 30-gigabyte hard drive can hold about 50 uncompressed CD albums. The lossy algorithms can raise this to about 500 CDs with only audio purists possibly detecting any loss of quality, or even a thousand with what many people would consider acceptable quality.

Lossless algorithms would bring this number to at least 80 albums with no loss of sound quality (no loss of information). Some reach average results as high as 150 albums, depending on the musical or audio content. There are many programs written for doing this and some will be examined here.

Typically, compression algorithms examine symbols in a data stream each in whole—whether those symbols are characters, bytes, pixels, or audio samples—and then look at that symbol's neighbors for some pattern or redundancy. This paper proposes an approach that examines the data stream in larger pieces, treats those pieces as arrays of bits where each row represents one symbol, and examines the arrays column by column—down the side—rather than one row at a time. It will be called *SLAC* for “sideways lossless audio compression.”

Audio vs. Data Compression

Professional audio engineers, sound editors, or music or film sound mixers are familiar with the term compression but they use it very differently. For them, “audio com-

pression” means that the dynamic range, the difference between the loudest and softest sounds, has been reduced. This is a critical step in the preparation of audio for the consumer. Natural sounds have a far too wide dynamic range for typical playback. Many electronic devices have been created to manipulate an electronic audio signal, each having their own characteristic *sound*. Audio compression algorithms used to act on a digitized version of an audio signal are often designed to mimic the characteristics of favored electronic compressors. That type of compression will not be covered further.

Objective

Lossless audio data compression has been an interest of this author for many years. As an understanding of the computing issues involved grew, the need to try out some of these ideas also grew. This new algorithm, SLAC, will be discussed along with the current state of lossless audio compression in general. Digital audio for consumers is the focus, but digital audio is just a special case of signal processing. The techniques tried so far, what their results seem to be, and some ideas for improving the output of the program will be covered. Of course, improvements mean smaller output files as this is about compression, and a faster running program. Techniques to maximize compression with this new view will be explored. A set of sound files was found on a web site that analyzes several lossless compression algorithms. These files will be examined with this algorithm and recompiled versions of existing algorithms, with the results extending what was found on that web site.¹

Basic Data Compression Techniques

Quite often the most efficient way to represent a symbol for easiest access and interpretation of what that symbol means is not the most efficient in the amount of space used.² Today, the most common encoding of text is called UNICODE. There are many variants for different languages and dialects. For demonstration's sake we'll talk about the very similar and older ASCII (American Standard Code for Information Interchange). It has the amazing property in that it uses the lower seven bits of an 8-bit byte to encode the alphabet in... alphabetic order. This is a silly statement, but a common algorithm is sorting, and the most common way to sort text is in alphabetic order. So it behooves us to use an encoding in which the natural order corresponds to the order of the data it is to represent. ASCII further applies a pattern to the letters and numbers to make them easier, thus faster, to work with. For instance observe how letter case is represented for upper and lower case "A":

```
A = 01000001
a = 01100001
```

By simply masking off the bits in the box, or even the upper three bits, a sorting algorithm that is to ignore letter-case need only continue as usual. And the numeric characters (0–9) are represented so that the four most significant bits can be ignored and the remaining bits are stored in two's complement encoding, which is the same way integers are most often encoded. This way they can be interpreted as an integer with a minimum of manipulation.³

With real world data, files will contain much redundancy and thus, wasted space. For instance, we can easily see that files that contain only ASCII numbers would be wasting (at least) half their space, as the four most significant bits aren't needed. In

regular text files there is rarely an equal number of each character so with more complex encoding techniques a variable number of bits can be used to represent each character with shorter strings of bits used for the characters that repeat most.⁴ Patterns that repeat can also be marked and pointed to later as that pattern reoccurs.

A key word that comes up in much of the literature on data compression is *entropy*. The entropy E of a symbol is defined by the probability P of the symbol being used (1.0 being all the time, 0.0 being none of the time, 0.5 being half the time):

$$E = -P \log_2 P .^5$$

The entropy of an alphabet is given as the sum of the entropies of the symbols in the alphabet. From this information we can do a statistical analysis of the data in a file and predict the best possible compression for the file.

Most often in the literature, the compression encoding type is considered to be either *statistical* or *dictionary*. Others group them differently.^{6,7} This author considers *transforms* to not be directly part of the compressing of data but useful in preparation of the data. For instance, taking the delta-transform or relative encoding on a stream of 32-bit floats may shrink the magnitude of the values stored in those floating-point variables but they will still take up 32 bits until one of these encoding types is applied to the data.

An implementation of some sort will likely employ several of these techniques tailored to best compress a certain type of data. One standard for FAXing documents encodes it with a predetermined table of statistics for the amount of white and black space represented as run-length encoding (RLE), with these counts represented by a variable-length code.⁸ The use of that predetermined table obviates the need for a document to be scanned twice on the transmitting side, or for the receiving side to build a table from the

received data in order to rebuild the image. It is important to know the needs of the data before making a blanket rule of how to represent that data.

Often file types are built around an encoding. The file type TIFF (tagged image file format) has a version of the RLE and LZW (Lempel, Ziv, and Welch; described later) techniques as standard encoding options to make storage smaller. Files with these options set will, of course, take longer to read and write than files without these processes.

Run-Length Encoding

Sound, picture, and even text files often have runs of repeating characters or symbols. These can be spaces inserted to format text, a character or characters to indicate a black sky in a picture, or integer zeros for silence between songs on a CD. Simple graphic pictures with long expanses of the same color are well suited for run-length encoding. It is considered a statistical encoding method but it's described separately here because it considers only data adjacent to the current value being examined. The remainder of the file isn't used except to determine the end of the run of this particular value. The statistical occurrences of other values in the file have no impact on how the current value is represented.

The trade off with run length encoding (RLE) is that it needs some kind of marker to indicate if the next piece of data represents another run of symbols. This can be either an escape character, which also needs a way to indicate that this character is to be taken literally occasionally, or an extra bit can be added to each symbol that when set, could indicate that *this* value is the length of the run of the previous symbol. In both cases care must be taken to be sure that there are enough redundancies that the added bit on each symbol or added character doesn't actually make the result larger.

Statistical Encoding

Statistical techniques for compressing data work by replacing symbols represented by an equal-length code with symbols from a variable-length code whose lengths are inversely proportional to their probability. Variable length codes only work if they follow the *prefix property*. The prefix property holds that once a certain bit pattern has been assigned as the code of a symbol, no other code can start with that pattern. Thus, no pattern can be the *prefix* of another.⁹ Ideally the shortest codes are then assigned to characters with the greatest probability. These algorithms are also referred to as “entropy encoders.” The simplest variable length code to visualize is a *unary* code. A version is shown here:

```
0 = 0
1 = 10
2 = 110
3 = 1110
4 = 11110
```

... and so forth. A count of zeros followed by a terminating “1” could also be used. One good thing about this code is that we don’t need to know in advance how many different values need to be encoded. This is because the numeric value indicates the count of 1’s to use before we reach a zero.

But the statistics of a character matter for compression. It would be foolish to encode a 7 as 11111110 when there are no 5’s or 6’s needing to be represented with the new code. Right away we can see we’d be better off encoding 7 as 111110 and save the space. Even more importantly, a string of these unary bits can be assigned to any symbol, such as the number 17, the letter Q, or even a string of characters like alphabet soup.

Several techniques have evolved and been refined over the last sixty years or so. This started with the work of Golomb, Rice, Shannon, and Fano. The most popular of these algorithms is Huffman coding, which always produces the optimal prefix-code for a given entropy.¹⁰ Rather than drawing out the steps, as is done in many texts, there is an excellent web site with a Java applet by Woi Ang where one can watch a Huffman tree built graphically, step by step at:

http://www.cs.auckland.ac.nz/software/AlgAnim/huffman.html#huffman_anim.

The Huffman code is so good because it builds a proper prefix-code from the frequency of symbols in a data set. This can be built for each file to be compressed (which is time consuming) or can be a canonical code based on typical usage. A large amount of standard text for a language can be examined by the programmer and a fixed Huffman encode and decode tree can be written into an implementation of the algorithm. There is also a technique of building the encode/decode tree as data is examined. This makes for faster encoding and decoding. As the algorithm progresses through a file a tree is built and the code is modified, as data is read and the frequency of the symbols changes. As an interesting note, the Huffman algorithm will build a unary code if the statistics of the data indicate that this would make the best encoding.

An important but difficult to understand statistical technique is *arithmetic* coding. An oversimplification of this process starts by mapping the symbols to encode over the interval $[0,1)$, with the size of the sub ranges for each symbol proportional to their frequency in the original file. When a sub range is used, it is broken into new set of sub ranges proportion to the original range of $[0, 1)$. This continues breaking down into smaller and smaller pieces, each represented by a number of greater and greater precision. This can produce better results than Huffman encoding by representing a large

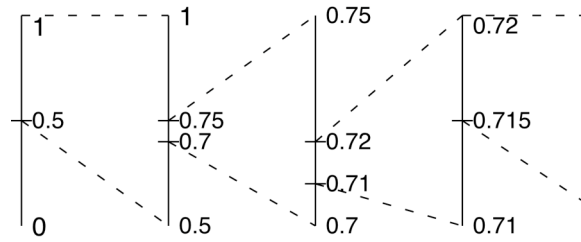


Figure 1. Recursive mapping of a range.¹¹

number of input symbols, or even an entire file as one very large number, and that each symbol, in effect, will have a different number of bits representing it at different moments. This is because the input symbol is represented by the range in which the stored number falls, rather than the bits directly.

On decoding, the range in which the number appears determines what symbol it represents, as each subsequent sub range determines each subsequent symbol. Let's say the stored value is 0.718, and a set of mappings where the three most numerous characters are represented by a , b , and c . The symbol a , having a probability of 0.5, is given the range $[0.5, 1)$; b , having a probability of 0.1, is given the range $[0.4, 0.5)$; and c , having a probability of 0.2, is assigned the range $[0.2, 0.4)$. Figure 1 shows how it maps the first four symbols of a set of data as a , b , c , a , etc.

Dictionary Encoding

Compression relies on files having redundancy. This redundancy need not only be based on the entropy of the characters used in a language but also on strings of characters repeating. A dictionary style algorithm will note where a pattern repeats and store a pointer to that repetition. Storing a pointer requires that the output data use more bits to represent a symbol so that we can discern when this symbol represents an original symbol or represents a series of symbols that have already been seen.

A popular algorithm (and the best until recently, see GZIP below) is LZW compression, named after its developers Lempel and Ziv, with later modifications by Welch. It works well with text and executable code.¹² It looks at files as strings of eight-bit symbols and outputs 12-bit symbols. A table with 4096 (2^{12}) positions is built as the input file is read. The first 256 output values are mapped directly from the input values. As a character is found on the input, it and subsequent characters are checked against existing entries starting at the end. If a match is found, the position of that match is written to the output rather than that character. If not, the single character is written as a 12-bit symbol and this pair is added to the table. As the table is built, the added “pair” can consist of a character, or reference to a string of characters, plus the one additional character. If the table fills up, it’s cleared and a new table is started.

Lossy Compression

It should be noted that this paper is primarily about lossless compression. There is “lossy” compression, with popular forms being JPEG for pictures and MP3 for music. These algorithms’ job is to find and throw out less important information to help make the resulting files smaller. They also have settings that an end user can adjust when the desired trade off between the resulting file size and the quality of the content have been determined.

JPEG

As we will examine later, a typical first step for signal compression is to increase the redundancy in the values representing a signal. A simple way to do this for audio, but not the best, is to take the running difference, or delta transform on consecutive pairs of values. A better way to correlate pairs of values, or pixels, for images when the desire is

to throw out information, as in JPEG compression (Joint Photographic Experts Group), is to map each pair's values as x and y coordinates in a two-dimensional space, rotate the result 45° toward the x axis, then read the new x and y values. This results in the y values grouping around one common value, zero. As we've seen before this lends itself to better entropy encoding. It also lends itself to better lossy compression as some of the bits to represent the accuracy of these smaller magnitude values can be thrown out with a smaller effect on the reconstructed image than if we hadn't taken this step.¹³ This has the visual effect of softening the edges of details in the image.

MP3

Besides the desire to store more music, widespread music sharing has increased popularity of the MP3 file type, or more accurately called "MPEG-1 Layer-3." This complex algorithm and those like this are sometimes called *perceptual compressors*. (There are many other perceptual encoding algorithms used for music and movies, and change depending on delivery method. Some are more transmission friendly, while other have a better sound quality.) It finds redundancy in material based on many psycho-acoustic studies and concepts that are used to determine what information can be thrown away, similar to the techniques used for JPEG. These algorithms break a signal into overlapping pieces, analyze the frequency content of these pieces, and store that information. This then allows the use of a psycho-acoustic technique called *masking*, as seen in Figure 2. Masking is the property that a louder tone will mask the perception of quieter tones that are close to it in frequency and time.

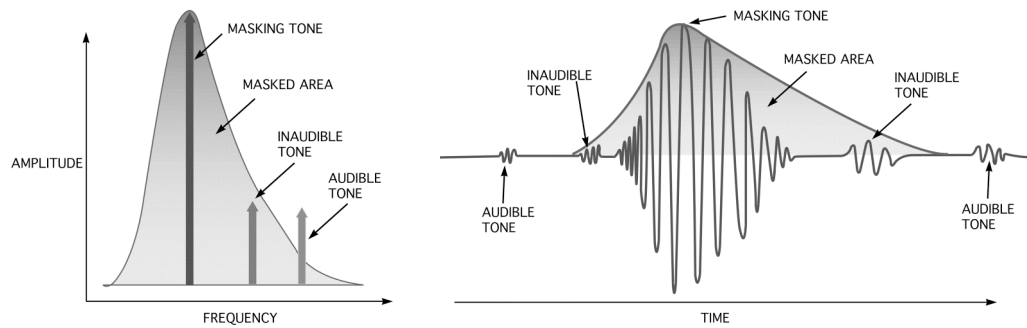


Figure 2. Effects of masking.¹⁴

A feature of these algorithms is that their bit rates can be set in advance, so they will only throw away the amount of data necessary to maintain that rate. They can choose to include all, part, or none of these other frequencies as needed, or to use fewer bits for marginal frequencies. These algorithms are also designed to pay more attention to (use more bits with) frequencies around 4kHz, as the human ear is more sensitive to these frequencies.

Lossless Signal Compression Programs

There are several lossless audio compression programs. Two of them, WavPack and FLAC (Free Lossless Audio Compression) were compiled locally for a proper comparison to the current incarnation of SLAC. GZIP was also tested for demonstrating the difference from those programs optimized for audio. Where information could be found, all of the signal compression algorithms use the same basic techniques (except for SLAC, of course). These can be broken down in two steps: transform and encoding. Correlation and prediction are both transform steps and both are used to shrink the magnitude of the values stored. Smaller values are more likely to have larger entropy. This is important for the encoding step. Encoding is then done with an entropy method. All programs, where information was available, use variations of Golomb-Rice or Huffman algorithms, or similar techniques of their own devising. FLAC adds the detection of long runs of absolute silence (sample values of zero) and run-length encodes these.

This excerpt draws attention to the architecture of many lossless audio codecs:

- **Blocking.** The input is broken up into many contiguous blocks. [...] The optimal size of the block is usually affected by many factors, including the sample rate, spectral characteristics over time, etc. [...]
- **Interchannel Decorrelation.** In the case of stereo streams, the encoder will create mid and side signals based on the [sum] and difference (respectively) of the left and right channels. The encoder will then pass the best form of the signal to the next stage.
- **Prediction.** The block is passed through a prediction stage where the encoder tries to find a mathematical description (usually an approximate one) of the signal. This description is typically much smaller than the raw signal itself. Since the methods of prediction are known to both the encoder and decoder, only the parameters of the predictor need be included in the compressed stream. [...]
- **Residual coding.** If the predictor does not describe the signal exactly, the difference between the original signal and the predicted signal (called the error or residual signal) must be coded losslessly. If the predictor is effective, the residual signal will require fewer bits per sample than the original signal. [...]¹⁵

GZIP

Even though GZIP is not a signal compression program, per sé, it's included as a good standard program often used for other types of data. As good as LZW is and all of the struggles with copyrights that are placed on it, it turns out that for most data GZIP compresses better than LZW.¹⁶ It's a combination of an earlier algorithm, LZ77, which uses a sliding window over the data to find duplications and redundancies, and the application of a predetermined Huffman table. The algorithm uses this window, which is up to 32Kbytes long, to search backward from the current byte for the longest match. It limits the length of the match and also takes the earliest match in order to produce the smallest reference value. This helps with the entropy and improves the effect of the Huffman algorithm. To improve execution speed, references to the past matches are placed in a hash table.¹⁷

Signal redundancy

In experiments, applying one of the simpler decorrelation techniques to a sound file before applying GZIP always resulted in a smaller file than using GZIP alone. This demonstrates how the requirements of compressing signals are different from other types of data, as mentioned in the abstract. If a predictive transform (correlation algorithm; discussed later) is applied to the data, the change from symbol to symbol can become very small. With the output from the delta transform, there will still be long sets of data where it will look like the data from a transform of the slightly louder data. This is also more in relation to how the data is perceived.

A program like GZIP, without a previously applied correlation transform, will see even a small moment-to-moment change in loudness as a completely different set of data

and never relate it to earlier versions of an otherwise identical waveform. Dictionary methods aren't going to be suited directly to signal compression.¹⁸ (See also "SLAC: Manipulating the Signal: Possible Transforms: Pattern Matching".)

Information in a signal is more dependent on how data in one moment is *related* to data in the next moment. Defining a "moment" is an important part of understanding a particular signal and how to compress it.

Transforms

Though transforms are not considered by this author to be a compression technique directly, transforms *are* important in the preparation of signal data before application of one of the compression techniques given above. Transforms act on how data relates to itself from one moment to the next, rather than the statistics taken and mapping done of the member values as done in non-signal compression algorithms. Study of lossless signal compression techniques shows that some transform of the data is done in each algorithm with the precise techniques kept secret in the highly competitive, closed source applications.

A transform can be as simple as a delta transform or running-difference, which means we store the difference between successive samples. We could also say "we predict this sample to be the same as the last." This works for signals as the values in successive symbols or values are closely related¹⁹ (also see "SLAC: This Approach" in this document). A slightly more sophisticated method, described later in this document, stores the difference between the current sample and the linear prediction carried from the last two samples. An algorithm can use recursive or convolution filters, or be as complex as

the Fourier Transform, or more commonly, the discrete cosine transform (DCT and variations) which is most often used in lossy algorithms.

A paper from Hewlett-Packard covers many details of what it calls *intra-channel decorrelation* and describes this part of the process this way:

The purpose of [this] stage of the typical lossless audio coder ... is to remove redundancy by decorrelating the samples within a [block]. ... Most algorithms remove redundancy by some type of modified linear predictive modeling of the signal. In this approach a linear predictor is applied to the signal samples in each block resulting in a sequence of prediction error samples. The predictor parameters therefore represent the redundancy that is removed from the signal and the losslessly coded predictor parameters and prediction error together represent the signal in each block.²⁰

Manipulating sound files

Programmers are less familiar with the way audio is stored than that of other files. We'll survey several here to familiarize the reader with these formats.

Sound Designer II (Sd2f)

Of all sound file formats this one is the simplest to use. It was created and made available by Digidesign, Inc. and was made public in hopes of it becoming a standard. The metadata, the data that describes how the sound data is formatted, is stored in an extra part of the file made available by Apple Computer called the *resource fork*. This leaves the *data fork* to contain only the actual audio data. If it's desired to begin reading audio from the start of the audio, the file pointer need only be put at the head of the file.

Here is an excerpt from the Digidesign specification²¹ document:

Sound Designer II files store all sound samples in the data fork and all sound parameters in the resource fork. This is extremely convenient for sound data where the data fork may grow to a hundred megabytes or more. Regardless of the size of the data fork you can add, delete, and modify sound parameters at will without compacting the sound data or moving it around the disk (and extremely time consuming procedure if the file is 100 MB). In addition, you may add your own parameters to a file (as long as their resource types don't conflict with Sound Designer II's) while allowing the file to be read by both Sound Designer and your program.

The Apple resource fork can be thought of as a file system within a file. The exact position of the data from the head of the fork need not be a concern of the programmer. The OS handles reading and writing the labeled information. The labels are a combination of a four-character type code (i.e. 'Sd2f', 'RW1c'), a 16-bit integer ID, and an optional name string. Any four byte-values, represented by extended ASCII characters, is allowed in a four-character code, but Apple reserves the use of all lower-case letters.

The format predefines several resource types, only three of which are of interest here (and are required by the specification). The three are of type 'STR ' (note the space at the end to make four characters) and this type is used to store short strings. String ID 1000 is the sample size in bytes, string ID 1001 is the sample rate and shown as a decimal (44100.0000), and string ID 1002 is the number of channels. The use of strings makes the data human readable, and Digidesign's software always writes the resource name to the file, increasing its readability. A track from an audio CD converted to this file type would have a sample size of "2," a sample rate of "44100.0000," and the number of channels would be "2." An early Digidesign workstation actually used 20-bit samples padding the last four places with zero-bits to make a whole three bytes.

The data fork is so simply laid out that it's best to just quote from the specification document:

The data of a Sound Designer II file is stored in Two's Complement encoding. Byte one of the data fork is the first byte of sound data. The sound data is organized as interleaved samples (if more than one channel) of either 8 or 16 [or 24] bit samples depending on the value of the 'sample-size' STR resource (see below).

For example, a standard 16 bit stereo file would be organized as follows:

Left Channel sample #1
Right Channel sample #1
Left Channel sample #2
Right Channel sample #2
Left Channel sample #3
Right Channel sample #3
etc...

Audio Interchange Format (AIFF)

Apple originally designed AIFF to be used for moving information between applications and platforms but thought it useful and flexible enough for use directly by an application. It's based on a standard developed by Electronic Arts²². Only the Macintosh file system uses the two fork file type (*data* and *resource*) so all information is arranged

and stored in the data fork of this file type. The complete specification is available from Apple and many other sources. The design of the format is so clever and flexible that it's worth study in itself.

The file structure is based on a generic type called a *chunk*, which the documentation describes with a C-like language thus:

```
typedef char[4] ID;
typedef struct {
    ID      ckID;           // chunk ID
    long    ckDataSize;     // chunk Size
    char    ckData[];      // data in variable sized array
} Chunk;
```

The member called `ckID` of type `ID` is a four-character code. This is convenient for humans and machines as four characters can be a useful abbreviation and a machine can read it as one native 32-bit value. The member called `ckDataSize` indicates the number of bytes in the array `ckData`. This size does not include the size of the members `ckID` and `ckDataSize`, which totals eight. The specification for a chunk can include other nested chunks and the chunk type can be unioned with any other structure of data the programmer wishes to code. Chunks can be in any order, so the first eight bytes need to be used as-is so a program can determine what the data is and how big it is, and calculate where the next chunk starts (if not end-of-file).

The AIFF file specification makes some alterations to this nested chunk system by defining a special outermost chunk called the *form chunk* or *container chunk* and uses the layout:

```
typedef struct {
    ID      ckID;           // always 'FORM'
    long    ckSize;         // always size of file minus 8
    ID      formType;       // always either 'AIFF' or 'AIFC'
    char    chunks [];
} ContainerChunk;
```

The `chunks` member is where the list of other chunks and nested chunks are placed. The three required chunks are called the Version Chunk, the Common Chunk, and the Sound Data Chunk. Each have the four-character IDs of ‘FVER’, ‘COMM’ and ‘SSND’ respectively, and can have only one of each. The Version Chunk was added on the change from the AIFF to AIFC type so that future changes could be made to the file structure without having to change the four-character file type code again (from AIFC to something else). Their structures look like this:

```
typedef struct {
    ID          ckID;          // 'FVER' for AIFC files
    long        ckDataSize;    // always 4
    unsigned long timestamp;    // 0xA2805140 (version as date)
}FormatVersionChunk;
```

for the version information,

```
typedef struct {
    ID          ckID;          // always 'COMM'
    long        ckSize;        // always 18 for AIFF
    short       numChannels;
    unsigned long numSampleFrames; // == samples/channel
    short       sampleSize;     // bits per sample
    extended    sampleRate;     // roughly a 10 byte float
    // below added for AIFC, makes struct variable sized
    ID          compressionType; // registered 4-char code
    pstring     compressionName; // human-readable type name
}CommonChunk;
```

to hold the audio data format, and

```
typedef struct {
    ID          ckID;          // always 'SSND'
    long        ckSize;        // the size of soundData + 8
    unsigned long offset;      // rarely used...
    unsigned long blockSize;    // ...set to zero
    unsigned char soundData[];
}SoundDataChunk;
```

to hold the sound data itself.

The sound data in the `soundData` member is in the same arrangement as the sound data in the “data fork” of the Sound Designer II file type above. Note that AIFF

stores the depth of each sample in bits where the Sound Designer II type uses bytes. If a non-multiple of eight bits is used for a sample zero bits are added on the right (LSBs) to pad it to the next 8-bit boundary.

The file type AIFC extends the AIFF type to include compressed audio data²³. A program needs to check the compression type `ID` in the new Common Chunk to see if it understands this compression type, if any. (The compression type could be set to `'none'`.) Compression type codes should be registered with Apple to be sure there is no conflict with other codecs.

Resource Interchange Format (WAVE)

This format was created jointly by Microsoft Corporation and IBM Corporation, and is very similar to the AIFF format as it is also based on the original Electronic Arts Interchange File Format.²⁴ The primary difference between this format and AIFF is that a WAVE file uses the Intel byte order and AIFF uses the Motorola byte order to represent numbers and audio samples. The four-character codes used to describe the different chunks are also different from the AIFF format and the structure and nesting of the various chunks is different.

The first four bytes of a WAVE file are `'RIFF'` and stand for Resource Interchange File Format. The specification also has a file type of `'RIFX'` where the bytes are stored in Motorola byte order. Then the next four bytes make an unsigned integer indicating the number of bytes in the rest of the file. The WAVE "Format Chunk" corresponds to the AIFF "Common Chunk," and the WAVE "WAVE Data Chunk" corresponds to the AIFF "Sound Data Chunk."

ID3

The ID3 tagging system is a standardized and flexible way of storing information about an audio file within itself to determine its origin and contents. The information may be technical information, such as equalization curves, as well as related meta information, such as title, performer, copyright etc.²⁵ Use of these tags has become very common in files created by consumer music librarian programs like Apple's iTunes. Professional software for creating MP3 files also often has fields for entering this additional information. It was designed so that it could be added to any audio file and merely be ignored if the program reading the file doesn't understand it. Even though the AIFF file format already has space for textual information, ID3 tagging was added by creating an ID3 chunk type 'ID3 ' and placing all the ID3 tags there.

ID3 tags are mentioned here because they are important additional information for identifying a file. This information occupies such a small percentage of the file that they do not need to be addressed as part of the compression. In fact, they should be left as is to help with further identifying the contents of a compressed file without decompressing or decoding first.

SLAC

Audio Editing & The Shape of the Waveform

The launch of cheaper, professional sound processing cards for personal computers in the early '90's was a tremendous advantage for smaller music and film production facilities, and for individual freelancers and hobbyists. Processing power and creative flexibility could now reach the hands of the average user. Professional quality audio here means that an audio channel, as an electrical signal, is sampled or measured 44,100 times a second and 16 bits is used to store each sample as a two's complement, signed integer. This is still the format of the standard audio CD. Before that, digital audio in personal computers was typically sampled and stored in 8-bit samples (one byte per sample) taken at between 8000 and 22,000 times a second.

From that point on, *seeing* the waveform of the audio signal by using software to access the card and manipulate the audio files was as common as—perhaps more common than—*hearing* the sound itself (see Figure 3). It was apparent immediately to this user that there was lots of white space above and below the peaks of the waveform, or zeros, used in representing a sound. An understanding of how audio is represented in digital form soon followed and with that a sense that there must be a better way to represent the data with storage requirements being so high. Figure 3 shows almost 1.5 megabytes of data in its approximately 16 second length.

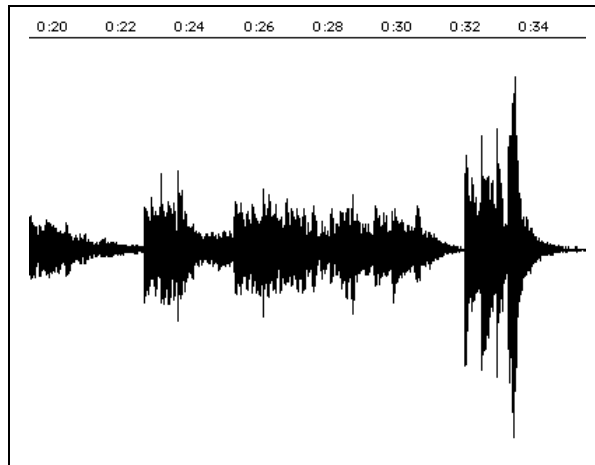


Figure 3. Portion of an audio file shown in audio editing software.

It became part of the sound editors' wisdom to not bother compressing digital audio with off-the-shelf compression programs (such as StuffIt) as one would with other computer data when archiving or backing up a project. Space reductions of 50% are considered a typical average for most data, with text files reducing to as little as 10% of their original size. Sound files, on the other hand, typically only compressed to 95% of their original size... hardly worth the time to compress and decompress the file which were generally very large and time consuming to handle to begin with.

It should be noted that that ratio of 95% was found in a casual test performed in 1994 with StuffIt, a generally free compression program for the Macintosh computer platform. It can be seen in Appendix E that the compression ratios are better now. Also note that the current version of `gzip` is used in that table and it gives better results than StuffIt.

Occasionally sound files would be examined with a hex-editor and patterns would be noticed in the data. Quite often every other byte was seen to be zero and these were noted to be quiet passages. This author's casual study revealed that standard compression

programs are tailored for text, thus the very small desirable results for text files, and not working as well on data where, for instance, every other byte could repeat.

At the time, it was assumed that the values in an audio file were mostly random with some weighting to smaller magnitude numbers. But the values in an audio file aren't random in their placement except for the sound of some disbursement of noise. As stated earlier, one sample of audio is generally closely related to the value next to it. An idea struck this author early on to transform the audio file by saving only the difference between pairs of samples, often called a delta-transform or difference engine. Applying StuffIt to this transformed file resulted in a compressed file that was about 70% of the original size (even better with `gzip`, see Appendix E). Later, the technique one author calls “delta encoding”²⁶ was found and is often used as part of a signal compression algorithm.

Class work for this degree introduced me to several computing concepts and algorithms that otherwise would have been considered too difficult. Comp 222 Computer Organization introduced Huffman encoding and showed how it can be used to assign shorter bit patterns to more common symbols. This also encouraged me to look again at a Macintosh ToolBox (system API) routine called PackBits, which does run-length encoding on a sequence of bytes (not bits as the name implies) to see if something could be done with that.

By just looking at the bits in an audio stream, it was decided that beyond the most significant bits staying the same for a large number of samples, less significant bits seemed to change at random, or at best, there would not be enough weight given to a subset of symbols to favor shorter strings of bits over longer ones. Study of others' attempts

show that an audio signal or other signals can be manipulated to favor smaller values. Where this is good for symbol-by-symbol encoding, it also increases the length of a run of the same bit in the left columns, which kept run length encoding as a more favored approach for this author. For a while, doing something like a Huffman encoding had been set aside until research for this paper revealed that most of the algorithms use a version of Huffman encoding.

After playing with some ideas in January of 2005 it was decided to “Google” for more information and a very good website was found where Robin Whittle²⁷ and another²⁸ compare in great detail several lossless audio compression programs, several of which have source code available (FLAC, WavPack, Monkey’s Audio, True Audio). Whittle made the original WAVE files he used for testing available on his web site so it was decided to also use those files in testing this algorithm. FLAC and WavPack were chosen for local testing. Reasonable comparisons can be made and conclusions drawn without repeating all of the tests with the other algorithms by extrapolating from the chart on Whittle’s web site.

This approach

Compression of digitized signals in general, whether lossy or lossless, takes advantage of the feature where one sample or one pixel is closely related to its neighbors. Suppose a signal has been digitized into signed 16-bit integers. We can visualize this as an array of bits where each row is a sample. Say there’s an array of samples starting at some value (see Table 1a with runs of 2, 4, and 8 in boxes) and count up one for each sample. Notice that starting on the right column we can see that the bits alternate every other row, then have run lengths of two in the second column, and have run lengths of

```

0000000010100000
0000000010100001
0000000010100010
0000000010100011
0000000010100100
0000000010100101
0000000010100110
0000000010100111
0000000010101000

```

(a)

```

...
0000001000111110
0000001010001000
0000001001101001
0000001011010111
0000001011011110
0000001010110011
0000001100001101
0000001110001010
0000001101110011

```

(b)

Table 1. The numbers 160–168 (a) and excerpt from audio file (b), both in binary.

four in the third. You’ll notice that most of the columns are correlated; that is, they have the same value throughout.

Notice this similar behavior in the short audio selection in Table 1b. The most significant bits are on the left and the least on the right. The bits in a column, primarily the left, repeat vertically. Intuitively, it can be seen here that runs of zero-bits or one-bits in a column are very common, so simple run-length encoding would be the most appropriate to use. Quiet passages in the program material or even the space between peaks of a waveform have lots of repeats. (Table 1b shows data from a quiet passage.)

The question becomes what is the best way to represent this data. Varying the number of columns examined at a time is considered. Each grouping requiring its own variation of the data structures.

It was taken *a priori* to start with the bits examined in pairs: 16th & 15th column first (leftmost, most significant bit is the 16th), then the 14th & 13th, etc. Other quantities and combinations of bits should be examined and tested. Two at a time were chosen after much thought (not experimentation, see Table 4 on page 28) as it seemed to be the most efficient way to group information.

Research on this paper led to the discovery that SLAC's method of examining the bits in a data set is similar to taking the bit-planes of an image. Different planes of bits in an image also show varying amount of correlation.²⁹ SLAC could easily be adapted to the lossless compression of images too.

The data types

Standard types

Over the decades, computers have had several groupings of bits used to represent some kind a symbol like a Roman character or an Arabic number, but now having a byte be eight bits as the core to represent one of these symbols is the standard. (The Unicode format has changed that, but this is a tangential issue.) From there grouping bytes in powers of two are used to create larger symbols. In the C programming language there is the “short integer” (or “short”, 16 bits) and “long integer” (or “long”, 32 bits). There's also the “long long” at 64 bits in length that won't be used here.

Additions for audio files

A convention for naming larger data types in audio files by Digidesign, Inc. will be used here. These types are variable in size, so these sizes are stored as part of a sound file to define what kind of data is stored and how it is to be interpreted. These terms are also used to define other sized pieces and types of data in various places such as audio CDs and electronic music keyboards. Clarity is needed here to avoid confusion with these similar uses of the same terms.³⁰

One slice or one instance of time of one channel of audio is called a “sample” and is typically one, two, or three bytes in size and interpreted as a signed integer in two's complement form. Other types can be used for a sample such as floats and longs but if

```

...
00000110010011110000011110010101
00000111101111000000101111001000
00001010101110010000111010001101
00001111000011010000111000001000
000100100100000000000110100100010
00010011111000100000110001111110
00010011100111110000110001100111
00010010011011110000110100001100
00010010011010010000110011110011
...

```

Table 2. Excerpt of audio bit stream with boxes around a byte, sample, and frame, respectively.

used at all they are for storage as the hardware to convert an analog signal to digital is generally converting it to integers of no more than 24 bits. Older personal computer hardware typically used 8-bit samples, the CD (Compact Disk) uses 16-bit samples, and pro-audio workstations (typically) use 24-bit samples.

A “frame” is one time-slice across all channels. Stereo, or two channels, will have 2 samples per frame and quadraphonic sound would have four samples per frame. The channel specifications for film sound of “5.1” and “7.1” would need six and eight samples per frame, respectively. A “7.1” channel, 24-bit recording would need 24 bytes per frame.

Data types for this algorithm

Two more data types will be used in this algorithm: a “block” and a “chunk”. A block will be a maximum of 2^{15} frames in size and will remain this size throughout a file except for the end of the file where the size of the last chunk is the number of frames in the file modulo the chunk size specified for the algorithm. The reason for 2^{15} will be come apparent below.

typedef struct zeroRepeat { short usePattern:1; short count:15; } zeroRepeat; // 0x	typedef struct bitRepeat { Byte usePattern:1; Byte useLiteral:1; Byte pattern:2; Byte count:4; } bitRepeat; // 10	typedef struct bitLiteral { Byte usePattern:1; Byte useLiteral:1; Byte count:6; Byte bits[64]; } bitLiteral; // 11
--	--	---

Table 3. The three chunk structures shown without extended comments.

The chunk will be one to 65 bytes long (see Table 3). These are pieces of encoded audio, each chunk representing some variable amount of audio data. The names were chosen because a “block” seems to suggest a uniform size like a “concrete block” and a “chunk” suggests a variable size like a “chunk of chocolate”. This is a different use of the term “chunk” from the Electronic Arts specification used at the basis for the AIFF and WAVE file types.

The chunk type is a union of three structures. In order to decode the data the first two bits of the byte are used to determine which of the following structures to use. Since the structures would never be used to indicate a run of zero, every value is offset by one like this:

```
00000000 = 1
00000001 = 2
00000010 = 3
00000011 = 4
```

... etc.

0x - Zero Repeat Chunk

This structure is used to store the count where a run from 17 to 2^{15} zero-bit pairs have been found in this pair of columns through this block of frames. The first bit of the chunk union being a zero, counting left to right, indicates the next 15 bits are to be used as the count of zero bit pairs:

```
typedef struct zeroRepeat {
    short usePattern:1; // must be 0
    short count:15;      // 32768 possible values, or 2^15
} zeroRepeat;           // size = 2 bytes
```

This is quite common with quieter passages of sound. A “short” can be used instead of this structure as values of 2^{15} or less (remember, values are offset by one) have the left most bit set to zero.

10 - Bit Repeat Chunk

A run of 4 or 5 to 16 bit pairs have been found in this pair of columns. (The “4 or 5” is explained in “bit literal”.) This can be any of the four combinations of two bits, including ‘00’.

The first bit of the chunk union being a 1 means to use this pattern, a 0-bit in the second position means to repeat this pattern, and the third and fourth bits store the pattern itself. The last four bits make the repeat count (maximum 2^4). The bits of the pattern to repeat are stored where they are as they will have to be moved in most cases to be placed back into position on decoding. Then the last four bits representing the count don’t have to be shifted to be used; the other bits can just be masked off:

```
typedef struct bitRepeat {
    Byte usePattern:1; // must be 1
    Byte useLiteral:1; // must be 0
    Byte pattern:2;    // 00, 01, 10, 11
    Byte count:4;      // 16 possible values, 2^4
} bitRepeat;          // size = 1 byte
```

11 - Bit Literal Chunk

The run of 1 to 64 *bytes* are to be used as is, no repetitions. Ones in the first two bits of the chunk union indicate this storage type. The last six bits are used as the count (1 to 2^6). Each byte will then hold the bit pairs in a pair of columns for four samples. A “repeat run” of four will be stored in a “bit literal” if we’re currently filling that data type

and it's not already full rather than switch to another chunk type. Since each byte can store four pairs of bits, the number of samples represented can be as high as 256. This data type could potentially add 128 *bytes* to represent a pair of columns literally as opposed to storing this data with out any counting.

```
typedef struct bitLiteral {
    Byte usePattern:1; // must be 1
    Byte useLiteral:1; // must be 1
    Byte count:6;      // 64 possible values, 2^64
    Byte bits[64];     // each stores 4 pairs of bits
} bitLiteral;         // size range from 2 to 65 bytes
```

Manipulating the Signal

These are transformations to put similar patterns of bytes/bits next to each other in a recognizable pattern for run length encoding. Research is showing that the real trick to doing lossless audio compression is to do some kind of manipulation to skew the data into a form that lends itself to patterns that are meaningful to the known lossless compression algorithms.

Complement and Rotate

It can be seen here *a priori* that runs of 0-bits in a column can be made to be by far the most common, so run-length encoding would be the most appropriate to use. It is easy to coerce the long runs of 1-bits in columns crossing negative sample values into zeros by taking the one's-complement of all but the sign bit of those negative sample values, thus guaranteeing that quiet passages in the program material or even the space between peaks of a waveform will have lots of zeros (see Table 4).

Taking the one's-complement of a sample, except for the sign bit, keeps most of the upper bits from changing when there are only a few bits changing across zero, then left rotate by one. This puts the sign bit, which is about as active as the least significant

Normal bit representation
in excerpt of audio in file.

```

...
0000010101010000
0000010101001100
0000010010100001
000001011111100
000001010100011
000000001011011
1111111100101010
1111110010110101
1111110101010001
1111110111011100
1111111011011100
0000000010101010
1111111100110111
0000000111101110
...
```



Same data with one's-complement
and rotate.

```

...
0000010101010000
0000010101001100
0000010010100001
000001011111100
000001010100011
000000001011011
0000000011010101
0000011010010101
0000010101011101
0000010001000111
0000001001000111
0000000101010100
0000000110010001
0000001111011100
...
```

Table 4. One's-compliment except sign of negative values, then rotate.

bits (LSBs) with the LSBs. Remember the signal is constantly changing and taking the running difference has the effect of hiding slow changing values leaving only the quickly changing values.

Once a block of frames residual code has been found by some correlation algorithm it may be desirable to find the maximum number of bits used in the samples. This number of bits can be encoded at the start of the output block, five bits give us a maximum of 32 values, rather than using up 16 bits for every pair of columns of zero bits.

Correlation and Prediction

Initially, only applying a delta transform was used in this algorithm. It's easy to understand and runs very quickly. The difference between samples makes more of the numbers smaller. This could potentially add another bit, but this would only happen for sections with very loud (high amplitude) high frequency program material. The worst cases for CD audio could be $32,767 - (-32,768) = 65,535$ and $(-32,768) - 32,767 = -65,535$ where each would take 17 bits to represent accurately. On the other hand this is

very easy to calculate as simple subtraction of neighboring pairs of numbers is needed while encoding and addition while decoding.

After some research WavPack was found to use, at its fastest setting, a slightly more involved equation. It's still very fast and its results are given in Appendix E. The equation is the difference of the current value minus the last value (stopping here would be only taking the delta) plus the previous delta:

$$out_n = in_n - \left(in_{n-1} + \left(in_{n-1} - in_{n-2} \right) \right).$$

So, this means we are estimating that the current value will be as much different from the last value as the last value was different from the one before it. The algorithm performs the transform in place in memory by reading the buffer from the end to the beginning, substituting zeros for the values when the index needs to extend before the beginning of the buffer.

Correlation between the channels of a stereo pair must also be considered. Primarily monophonic program material will be very similar in both channels and therefore considered to have redundant information. Resolving this requires taking the sum and the difference of the two channels. After processing, primarily monophonic material will be strong in the new sum channel, and the difference channel will be very quiet, therefore containing more and longer runs of zeros. Each block is tested to see if it needs to be left in *LR* (left-right) form or put in *SD* (sum-difference) form. Normal stereo files will either be unchanged or improve the compression possibilities by taking the *SD* of the audio file. Taking the *SD* will hurt the compression results for program material that is primarily heavy in one channel.

This could also potentially add another bit to the sample. This will happen for signals that go above $\pm 2^{15}$ in each of the channels for (16-bit samples). To return back to left and right channels, sum the data in SD for the left channel and take the difference for the right, the output must be divided by two which can be done quickly by an arithmetic bit-shift to the right. In these equations S is the sum signal, D is difference with

$$S = L + R,$$

$$D = L - R;$$

so going back,

$$L = \frac{(L + R) + (L - R)}{2},$$

$$R = \frac{(L + R) - (L - R)}{2},$$

which is shown with the original left and right substituted for S and D .

Encoding

Data is loaded in blocks of 2^{15} frames because this is the maximum value that can be represented with 15 bits. Thus the maximum number of zero-bits in a column and the reason for the design of the “zero repeat” chunk type. Since each correlation transform can require another bit to properly represent the data, all samples are promoted to longs so as to not over flow the storage of those numbers. (Eight-bit samples need only be promoted to shorts, the next native type larger.) If, say the 17th and 18th bits remain zero in a 16-bit sample after the transforms the only cost is the time to count the pattern and two bytes in the output file representing that this column of bits is all zeros. The majority of values being smaller far outweigh the occasional value that needs more bits to be accurately represented.

Next, one or more decorrelation algorithms are applied to the block and a flag is set in the output stream. Each block can have a different algorithm applied. Part of the analysis is to determine which algorithm or combinations of algorithms will produce the best results. The program tests if interchannel decorrelation is necessary. The decode step is faster as it only needs to choose the algorithm indicated by the flag. The last step before encoding is to one's-complement each sample and rotate it to put the sign bit on the right, as shown before in Table 4.

It was also considered that this file type could be used for streaming audio, thus the need to identify the start of a new block without having to start at the beginning of the file. It was determined that three sequential zero bytes would never occur as part of the algorithm and this is used to signal the start of a block. It should be noted that streaming broadcasting software requires audio to be encoded at a constant bit rate, which this is not.

For each block in the file, write three 0-bytes, then one byte with the correlation flags. Next save the size of the block (the last block might be short) as a short integer. Then for each pair of bit columns in the bit array, step through the entire block as indicated starting with the second state in Figure 4.

The working of the state machine in Figure 4 can be described like this: Scan for matching bits by marking where we are and grab a copy of the current pair of bits, then count the number of frames this pattern repeats in this pair of columns. If the repetition pattern is two zero-bits and there are more than 16 in a row, then save to the output file the count of zero-pairs in the "zero repeat" structure, which is the same as using a (positive) short integer. Runs of other bit patterns longer than 16 will have to be stored in

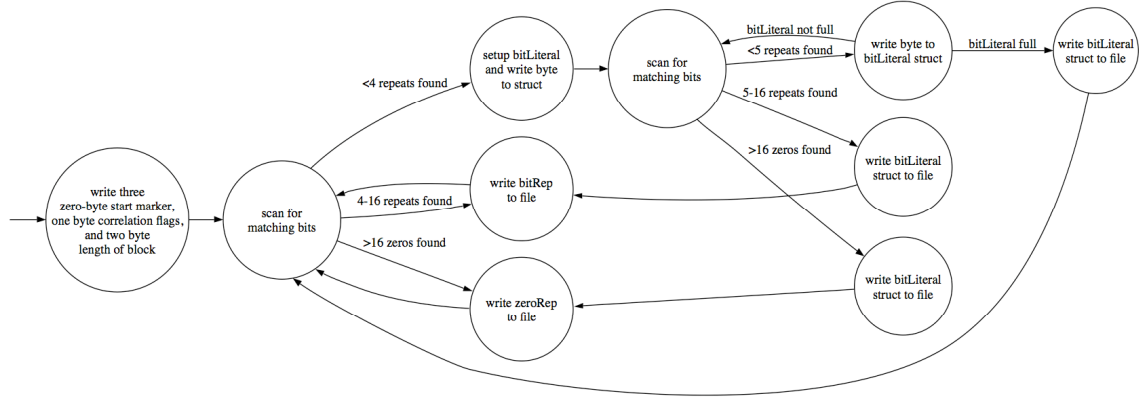


Figure 4. State diagram for encode algorithm.

multiple “bit repeat” structures, but not if there are 17 to 19 in the count. This will become clear below.

Other than long runs of zero-bit-pairs, repetitions of any pattern of pairs of bits with counts of four or five to 16 will be stored in the “bit repeat” structure. If runs are longer than 16 a new “bit repeat” structure will need to be used. This structure will never be used to store counts of less than four. So if the count modulo 16 is less than four then that amount will be stored in the next data type.

If the run is three or fewer, then four pairs of bits are copied from the next four frames using the “bit literal” structure. When the structure is full, or a scan indicates a different structure should be used next, it is written to the output buffer. Repeating patterns of only four pairs of bits are also written to this structure—if it already exists—because switching to the “bit repeat” structure also only takes one byte giving no space advantage. It could also require the immediate restarting of the “bit literal” which has as much as an extra byte for every 8 bits saved, adding potentially 1/3 more to the size of the output than an unprocessed file, in the worst case.

Code for saving the “bit literal” structure needs to be just after the pattern counting is done. If a run of three or fewer is found it’s either added to the existing “bit literal” structure or a new “bit literal” structure is set up. If there is a long enough repeating pattern found and a “bit literal” structure is in use then that “bit literal” structure is saved to the output buffer before writing the count of the current pattern to the output file as a “bit repeat” or “zero repeat” structure.

Decoding

The compressed data can be read from the file in any size piece at a time. The current Macintosh file system (this programmer’s system of choice) is most efficient if file reads are in 4k-byte chunks so the compressed file is read in multiples of that. The decoding of the data can proceed one byte at a time using the form of a state machine. The first two bits of each byte are used to determine to which state we advance. The current state is held in a variable that is tested by a switch-case construct. Not shown is a counter that returns the machine back to state one. The numbers in Figure 5 actually correspond to the byte number in the block we are examining. The transition labeled “10” goes back to the same state because the code to handle this condition is placed directly where it’s detected without changing the `state` variable. Actually changing states may be less confusing and may produce just as good compiled code given the current state of optimizing compilers.

The frame counter is tested to see if that pair of columns in the bit array have been completely set for that block to indicate when we move to the next pair. When all the pairs of columns are done, the entire block is written to the output file and the state ma-

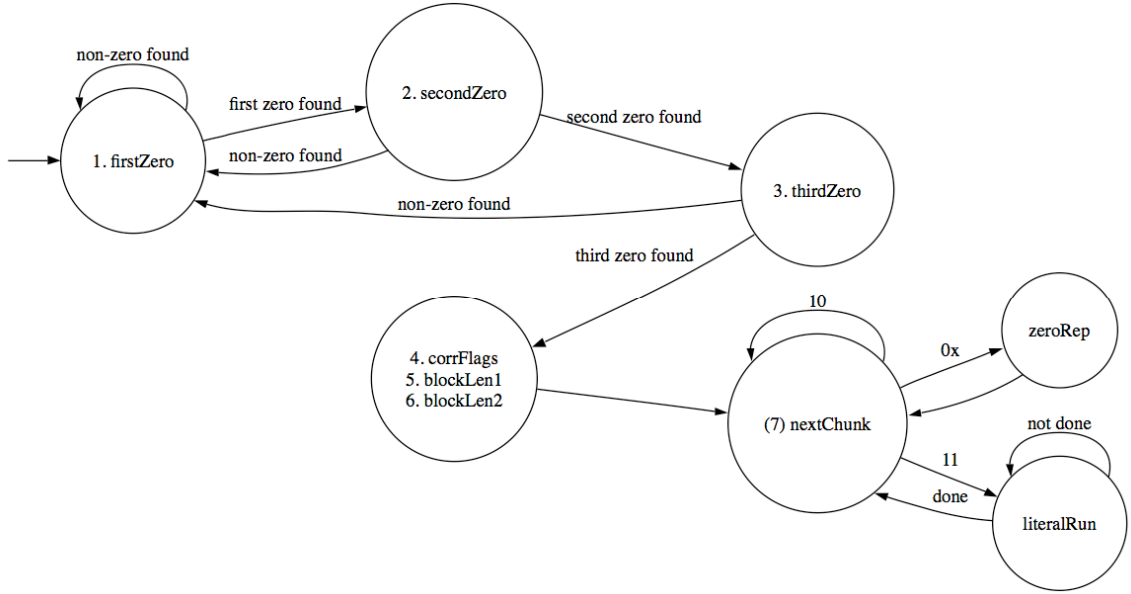


Figure 5. State diagram for decode algorithm.

chine is told to restart or exit depending on whether there is more data in the compressed file.

The machine states are identified by what we are looking for, so at the very beginning the state is set to “firstZero” as we are looking for the first zero byte of three that identifies the start of a block. When found it switches to the state “secondZero”. If not found it keeps looking (this could be a stream rather than the start of a file) by switching back to looking for the first zero-byte that will mark the start of a block. The next byte contains the flags for what correlation algorithms were used, then two bytes are used as the short integer that describes the length of the block and are copied to the frame size variable in the two states “blockLen1” and “blockLen2”. When that is done we set the state to “nextChunk” and start interpreting the chunks.

Why is the length of a block read in two steps? This is because there is a chance that the first byte of the short integer may be the last byte in the input buffer filled from

the last time the compressed file was read. The algorithm needs to be prepared to wait for the input buffer to be refilled before reading the next byte.

The state “nextChunk” signals we’re looking for the next chunk to interpret. Now, if the first bit is a zero—the byte masked to show only the first two bits be visible can have a 0x00 or 0x40—then we grab this byte as the high byte of a short and set the state to “zeroRep” to indicate we’re looking for the second byte of the zero-repeat structure. Once that’s found we switch back to “nextChunk”.

While in “nextChunk”, if the first two bits are “10” (0x80) then the next two bits are saved as the pattern to repeat and that pattern is repeated the number of times saved in the last four bits.

Lastly, if the first two bits are “11” (0xC0) then we put the last six bits into a counter to show how many of the next bytes to write out explicitly and switch to state “literalRun”. We stay in “literalRun” until the counter has been decremented to zero and we then switch back to “nextChunk”.

The output buffer could be overrun when the bitLiteral structure is written to it. A non-multiple of four frames could have been counted by the zeroRepeat or the bitRepeat structures. So even if there’s only one literal pair of bits to write it will write four pairs. It is easier to code and faster running to declare a buffer with three extra frames at the end than to constantly check for the overrun.

The state diagram is re-started by counters that are not shown or exited completely when the end of file flag is found by the function reading the file.

Storage format

The file format for storage of the output of this algorithm is based on the AIFC file format. The file format also allows for any type of compression to be used and can be any standard type or new type. It's up to an application as to whether it can handle that type or not. Meta data beyond sample rate, size, and number of channels can be copied in total to an AIFF style chunk. This prevents the need to understand the meaning of all the other metadata when coming from non-AIFF files. Of course if the original file is any type of AIFF or AIFC file then it makes sense to just copy all meta-data to the new file without change. Copying from the Sound Designer II format would require some special effort, as metadata is stored in the Macintosh resource fork.

Analysis

One audio CD was chosen for testing because of its familiarity to the author and its variety of selections. It's a demonstration CD put out by the company Brüel & Kjær in 1989 to show off their brand of microphones. This author has used this CD extensively for subjective testing of other parts of audio systems. The album includes a variety of pop, jazz, and many classical selections, along with sound effects and test signals. It was found to be useful that all of the selections were recorded with microphones from the same manufacturer, all of which are either a small or large diaphragm model of the same design. Where musical and performance styles vary widely the sound quality remains consistent from cut to cut.

The web site "Lossless audio compression"³¹ provided the inspiration to follow through with this lossless compression idea. On referring to it recently, it was discovered that Whittle had added the audio he'd used in his testing, making it available to download as WAVE files. This solved the problem of making comparisons to work he'd already done. Both sets of audio were used in testing as this writer wanted to see results on audio that he was familiar with and wanted to use data to make meaningful comparisons to Whittle's work. Whittle's web page also refers to test files which he did not include for download and the B&K CD has suitable selections on it. On that site Whittle gives a short description of his sound files including their source with reference numbers and average volume level.

The time needed to process a file was timed with a shell script that was set up to step through them. Originally the intention was to use `StuffIt` instead of `gzip` to compare with this author's previous experience of compression of digital audio. There is no com-

```

#!/bin/sh
for i in `ls "$1" | sed -e 's/ /:/g'`
do
    i=`echo $i | sed -e 's:/:/g'`
    echo $i
    #time -p nice -n -20 cp "$1$i" "$2"
    #time -p nice -n -20 gzip -c "$1$i" > "$2/$i.gz"
    #time -p nice -n -20 gunzip "$1$i"
    #time -p nice -n -20 slac "$1$i" "$2"
    #time -p nice -n -20 slacd "$1$i" "$2"
    #time -p nice -n -20 slacwp "$1$i" "$2"
    #time -p nice -n -20 slac -d "$1$i" "$2"
    #time -p nice -n -20 flac -0 --totally-silent "$1$i"
    #time -p nice -n -20 flac -8 --totally-silent "$1$i"
    #time -p nice -n -20 flac -d --totally-silent "$1$i"
    #time -p nice -n -20 wavpack -f "$1$i"
    #time -p nice -n -20 wavpack -h "$1$i"
    time -p nice -n -20 wvunpack "$1$i"
    echo ""
done

```

Table 5. Shell script used for testing.

mand line version of StuffIt and it was desired to keep the timing framework consistent between the tested algorithms.

In the shell script in Table 5 the UNIX `sed` utility in the second line is there to change spaces into colons because the Macintosh file system allows file names with spaces and disallows colons, but a shell script's "for" statement uses spaces as a delimiter between items to process and will break a path in the wrong place. The fourth line changes the colon back to a space so the desired process can find the named file. The script takes source and destination directory paths as parameters. Both SLAC and `cp` need source and destination parameters as they both either need a new name for the file or a destination directory. `gzip` options were used to create new files from the original sound files rather than deleting the originals.

It was the desire of the writer to use open source algorithms since compiler optimization could be made similar across all programs thus eliminating one variable in the behavior of the programs. Both FLAC and WavPack were compiled from source code on a 1.67GHz PowerBook running MacOS X 10.4.5 (gcc 4.01). The algorithms were run on that laptop with the sound files read from and written to the laptop's internal hard

drive. A working version of this author's algorithm was also compiled with `gcc` so it could also be used in the same timing script as the other algorithms. Primary work and debugging was done in Metrowerks Codewarrior, but it's not clear how well code optimization compares between Codewarrior and `gcc`, so `gcc` was used to keep down the possible variations. Macintosh ToolBox calls were adapted to the UNIX programming style rather than taking the time to rewrite them.

Timing was found to be very inconsistent between runs of any of the algorithms, varying upwards of 20% from run to run. It was determined that the cause of this was the fact that the host OS is a preemptive multi-tasking OS that could interrupt the algorithm at any time for other housekeeping. The OS is also known to have algorithms that cache and optimize executables³² so one pass through the set of files was repeated in reverse order to check for that influence. Nothing beyond the earlier noted variation in processing time was noticed.

Later tests were re-run with the UNIX utility called `nice` to give priority to the tested program. Negative numbers for the “-n” option mean to be less nice to other running programs, giving priority to the current program. The most stable performance from run to run was found when each program was given a “nice” value of -20 and the shell script executed from a new shell started with a “nice” value of -10:

```
sudo nice -n -10 sh
```

with most performance time averages staying within 10% of each other.

Table of results

The results shown in Appendix E need some explanation. All files were in AIFF format with their sizes shown in bytes. The program `cp` is the UNIX utility to copy files.

It's here to show the overhead a plain copy imposes. Time is always in seconds and every column labeled “encode” or “decode” is the time in seconds to encode or decode the file. The “t/M” column is the time normalized to process one minute of audio with this equation

$$n = t / (s / 10584000),$$

where n is the normalized time, t is the total time to process on that file, and s is the original file's unprocessed size. This size was chosen because half (or all, in the case of `cp`) of each process moves the original amount of data. The unlabeled columns after the “encode” and “decode” columns are the normalized times for those processes, too.

The main column group titles are mostly self-explanatory. “Delta” refers to the delta transform. In the script in Table 5 the programs `slac`, `slacd` and `slacwp` correspond in the tables below to `slac` with no transforms, `slac` with delta, and `slac` with WavPack's simplest decorrelation transform. The rest show their command line options, generally run twice using the fastest setting and then the strongest setting. The very last foreshortened columns are WavPack's strongest option with the files done in reverse order to check the effect of the OS caching commands.

Screen shots

Since this approach to signal compression was discovered by visually looking at the shape of the signal, views familiar to sound editors are shown in Appendix A.

Waveform

The term “waveform” is used for a time versus amplitude view of a sound file. The images here are screen shots of each selection of audio displayed in a ProTools “session” file. The x-axis is measured in time and is set to show the entire length of the

file no matter what the actual length of the file is. The timing marks were also captured for reference and perspective. It's assumed from experience with this software that each screen pixel width is an average (not peak) of the signal's positive values and average of its negative values for the length of time the pixel spans.

Spectrum

The term "spectrum" is used for a frequency versus amplitude view of a file. The spectrum for each file was taken over only about ten seconds of program material at what was thought to be a representative section of the whole file. The motivation for this was as a time saver and also knowing the utility taking the spectrum would show the maximum values, so only a section with a strong signal was needed.

The software is a demonstration version of the Waves Audio Ltd. software plug-in called "PAZ" and it was set to take RMS values. It is thought that as the RMS value for a signal is considered to more closely represent it's perceived loudness³³ and that this correlates to compression results. This is because a single peak in the signal could be construed as being representative of the program material, but one loud peak in the signal would have little effect on the compression of an otherwise quiet passage.

Stereo correlation

This part of the Waves plug-in is a more elegant version of what audio engineers did by taking a feed of the left and right program material and connecting it to the y and x inputs, respectively, of an oscilloscope. Sensitivity was adjusted to show the typical "ball-of-string" pattern, depending on the program material, but typical for reverberant material. A vertical line indicates left-channel only program material and a horizontal line indicates right-channel only program material. Program material that is identical in both

channels is shown as a line with a slope of positive one. A well-rounded ball-of-string pattern means that the program's material is well spread out in space and will seem to surround the listener.

The plug-in modifies this in several aesthetic ways but the information is the same. First, it tilts the display so that monophonic or center panned program material appears as a vertical line. Left and right program material show as slopes of negative and positive one, respectively. The scale of the display has been made to be logarithmic rather than normal linear view on an oscilloscope. It also has a peak hold function and averages the signal causing it to look more like a spiked ball rather than a wad of string.

In the following screen shots the vertical grey band in the images in the left column are a selected region of approximately ten seconds in length. This region was then used to generate the frequency response and stereo correlation images on the right.

Conclusion

The results are encouraging but the open source programs work much better for compression. SLAC *does* seem to have the edge on speed, performing better than the otherwise faster program, WavPack, with the same correlation algorithm. The future experiments mentioned below seem to be warranted.

Timing information was later gathered from the original Codewarrior compiled version of SLAC and was found to be comparable to the `gcc` compiled code. The effort to compile versions of the other algorithms locally in addition to SLAC may not have been necessary.

Future Experiments

Possible Transforms

Rather than just using delta encoding a more sophisticated approach could be used. It must be kept in mind that different transforms may add a bit or more to the resolution of the signal.

Some other transforms include linear predictive coding, wavelets, and discrete cosine transform. All of these have floating point and integer versions. The literature contains many and more sophisticated methods. Something simple may be more important than more data efficient algorithms as speed of processing is also a consideration when applying hardware to encode and decode the data.

All techniques need to predict a value only from previous values, or at least in the same direction as they are decoded. This way a transformed value can be derived from known values.

Pattern Matching

A better way of applying a running difference or delta to the signal might be to recognize a fundamental (as in harmonic fundamental) frequency and subtract the previous cycle from the current. This is similar to the LZW algorithm except that an exact match shouldn't be sought. It might be made to match all but the lower two or three bits and store a pointer to the pattern and the residual of the difference. The problems with this are determining what the fundamental frequency for a block is and in allowing for changes in that frequency.

Another possibility is to use a version of a dictionary method like LZW but again, to only match to within a few least-significant bits, then code the reference chunk and residual bits of the current set of symbols minus the reference chunk.

Splines

Several past points, or samples, can be used to determine a value based on spline curves. Earlier testing by this writer for splines to be used in decimation and interpolation algorithms showed that splines aren't good at representing signals, but the idea is to get closer to the actual next value with the simplest possible calculations. One of the spline techniques may prove to deliver a small residual code with a minimum of calculations.

Tunable Filter

A block can be tested for its dominant frequency. Then a band-pass filter can be tuned to that frequency and its output used to predict the next value with the difference from the actual value being stored. It may be useful to pair this with the output of the delta engine.

For that matter, a simple FFT (fast Fourier Transform) could be passed over a block of data and a handful of frequencies with their intensities could be encoded at the start of a block. This can be thought of as using a small amount of storage to indicate some redundancy. These frequencies are then used in the decorrelation step.

1-bit slices

In this first version of the algorithm it was assumed that the columns of bits would be best examined in pairs. The sign bit (most significant) is bit-rotated to the right (least

significant) position so it is grouped with bits that will be changing approximately as often, leaving the leftmost bits to be changing the least.

After discovering the degree of importance that correlation is to the success of a lossless compression algorithm, and that there are many stronger algorithms than the delta transform, it seems that examining only one column of bits at a time may prove to be better. This would require only two structures and obviate the need of the rotate step in the complement and rotate step. While encoding, the algorithm can scan the sign bit column first and then continue on with the other columns in any order.

The “zero repeat” chunk would be used as is—but, of course, it would be only referring to a run of one column in the bit array rather than two, and would continue to refer to counts of 17 to 2^{15} zeros. The other would be a modified version of the “bit literal” chunk. The first bit set to 1 would indicate that this structure is to be used and the next seven bits used to indicate the number of the next bytes to interpret literally. The structure would look like this:

```
typedef struct bitLiteralOne {
    Byte usePattern:1; // must be 1
    Byte count:7;      // 128 possible values
    Byte bits[64];     // each stores 8 bits from column
} bitLiteral;         // size ranges from 2 to 129 bytes
```

Now each instance of the new “bit literal” structure can hold twice the number of bits as the 2-bit “bit literal” structure. A situation where *two* entire columns of bits must be encoded literally would now only cause an extra 64 bytes to be inserted. This is another thing that may make this structure better: output of the current encode algorithm using the two bit structure often has many consecutive “bit literals.”

Lossy compression

A lossy form of this same algorithm could apply a certain amount of slew rate limiting, by limiting loud high frequency signals. This would let the running difference values stay smaller, but may be an objectionable form of data reduction.

Even better, use the example set in WavPack where two files are generated after the encoding. One file can be used by itself as lossy playback, or with the proper decoder, play both files where the second contains the residual error information, thus reconstructing the original signal completely.

Lossless with Another Lossy

As an experiment, convert a full fidelity, AIFF file to some lossy format. Then convert that small lossy file to a new AIFF file. Now subtract, sample by sample, the new transcoded AIFF file (with all its inaccuracies from passing through the lossy encoding) from the original file and call the new file the “residual” file. Next apply one of these compression techniques to the residual file. The combination of the sizes of the *lossy* file and the *residual* file may be the smallest lossless result yet. These two files could be woven together into one file.

Other Reading

Stuart, J. Robert “Coding High Quality Digital Audio” (Meridian Audio Ltd).

Rowe, Robert *Machine Musicianship* (Massachusetts Institute of Technology 2001).

Chamberlin, Hal *Musical Applications of Microprocessors*, 2nd Ed. (Hayden Books 1987).

Rorabaugh, C. Gritton *DSP Primer* (McGraw-Hill 1999).

Costa, Max H. M. and Malvar, Henrique S. “Efficient Run-Length Encoding of Binary Sources with Unknown Statistics” (Microsoft Research Microsoft Corporation 2003).

Endnotes & Bibliography

- ¹ Whittle, Robin, "Lossless Compression of Audio" <http://www.firstpr.com.au/audiocomp/lossless/> 1998–2005.
- ² Smith, Steven W. *The Scientist and Engineer's Guide to Digital Signal Processing*, 2nd Ed. (California Technical Publishing 1999): 481.
- ³ Smith: 69.
- ⁴ Salomon, David *A Guide to Data Compression Methods* (Springer-Verlag New York, Inc. 2002): 1–2.
- ⁵ Salomon: 9.
- ⁶ Salomon: 3.
- ⁷ Smith: 483–494.
- ⁸ Salomon: 32–40.
- ⁹ Salomon, David *Data Compression: The Complete Reference* (Springer-Verlag New York, Inc. 2000): 47.
- ¹⁰ Wayner, Peter *Compression Algorithms for Real Programmers* (Morgan Kaufmann 2000): 19.
- ¹¹ Salomon: 117.
- ¹² Smith: 488.
- ¹³ Salomon: 243–247.
- ¹⁴ Dipert, Brian "Digital audio breaks the sound barrier" *EDN Magazine* July 20, 2000, pp. 74–75
- ¹⁵ Airola, Cangialosi, & Dimino "PRESTO - Preservation Technologies for European Broadcast Archives" (PRESTO Consortium 2002) 10–11.
- ¹⁶ Salomon: 202.
- ¹⁷ Salomon: 205.
- ¹⁸ Salomon: 642.
- ¹⁹ Smith: 487.
- ²⁰ Hans, Mat; Schafer, Ronald W. "Lossless Compression of Digital Audio" (Hewlett-Packard Nov. 1999) 6–11.
- ²¹ "Sound Designer I and Sound Designer II File Formats" (Digidesign. Inc. 1989) 4.
- ²² 'Audio Interchange File Format: "AIFF": A Standard for Sampled Sound Files Version 1.3' (Apple Computer, Inc. 1989) 1.
- ²³ "Audio Interchange File Format AIFF-C: A revision to include compressed audio data" (Apple Computer, Inc. 1991) *i*.
- ²⁴ "Multimedia Programming Interface and Data Specifications 1.0" (IBM Corporation and Microsoft Corporation 1991) 10–37.
- ²⁵ "ID3 tag version 2.3.0" (ID3.org 1999).
- ²⁶ Smith: 486–488.

²⁷ Whittle.

²⁸ “Performance comparison of lossless audio compressors” <http://members.home.nl/w.speek/comparison.htm> 2005

²⁹ Salomon: 235–237.

³⁰ “Global Music Resource - Digital Audio Formats and Streaming Audio Formats”
<http://www.globalmusicresource.com/basics/basics3.html> 2003.

³¹ Whittle.

³² Singh, Amit “Making An Operating System Faster” <http://www.kernelthread.com/mac/apme/optimizations/> 2005.

³³ Smithers, Brian “Meter Matters” (Electronic Musician 1/2003) 3.